

世界模型的发展脉络：从 Dyna 到 Game World Model

2026 年 7 月 3 日

摘要

世界模型研究关注一个核心问题，智能体怎样在内部形成可预测、可规划的环境表示。在强化学习里，世界模型常用于学习环境转移、奖励与终止信号。在自动驾驶、机器人、游戏和视频生成研究中，世界模型又扩展到可交互的场景模拟、未来生成和长时序推理。内容按时间线梳理从 Dyna、早期 model-based RL，到 *World Models*、PlaNet、Dreamer、MuZero、TD-MPC，再到生成式世界模型与 JEPA 路线的发展脉络。同时补充 VAE、latent state、RSSM、planning 等底层概念，并进一步加入 DINO/DINOv2、具身智能研究版图、benchmark 以及 game world model 四个补强部分，用于建立从基础公式到研究问题的连贯直觉。

目录

1	核心问题与基本记号	3
2	从 Dyna 到深度模型	4
2.1	Dyna 的思想	4
2.2	MBRL 的一个长期问题	5
3	VAE：世界模型里最常见的底层积木	6
3.1	为什么需要 VAE	6
3.2	VAE 的核心公式	7
3.3	VAE 在世界模型中的作用	9
4	Ha 和 Schmidhuber 的 <i>World Models</i>	9
5	PlaNet、Dreamer 与 latent dynamics	10
5.1	RSSM 是什么	10
5.2	PlaNet 的位置	11
5.3	Dreamer 的位置	12
5.4	PlaNet 到 DreamerV3 的详细对比	12
5.5	latent space 与 latent imagination	13
5.6	Dreamer 的训练闭环	13

目录	2
6 MuZero 与 TD-MPC	14
6.1 MuZero 的抽象模型	14
6.2 TD-MPC 的控制视角	15
6.3 Dreamer、MuZero 与 TD-MPC 的路线差异	15
7 Transformer、视频生成和可交互环境	16
7.1 序列建模的视角	16
7.2 IRIS、GAIA-1、DriveDreamer、Genie	17
7.3 常见实验环境：Atari、Minecraft 与 ViZDoom	17
7.4 生成式交互环境工作对比	18
8 JEPA 路线	18
9 DINO/DINOv2 与非生成式视觉表征	19
10 具身智能研究版图：世界模型、VLA 与 LAM	23
11 Benchmark：具身智能进展如何被衡量	24
12 Game World Model：从游戏智能体到神经游戏引擎	25
13 一张时间线	27
14 阅读路线	28
14.1 第一层：世界模型主线必读	28
14.2 第二层：视觉表征和具身智能补强	28
14.3 第三层：benchmark 和评测	29
14.4 第四层：game world model 重点阅读	29
结语	29

1 核心问题与基本记号

如果把一个智能体放进环境里，它每一步都会看到观测、执行动作、拿到奖励，并进入下一时刻。世界模型研究希望回答的问题是：

- 环境当前长什么样。
- 智能体做出动作以后，环境会怎样变化。
- 哪些内部表示足够稳定，可以支撑规划和控制。

先说明一个容易困惑的地方：下面这些变量可以用来抽象世界，来源有三层。第一层来自物理和因果直觉。动作会改变环境，环境变化会产生新的观测，任务目标会通过奖励体现出来，回合是否结束需要单独记录。第二层来自状态空间模型和马尔可夫决策过程。控制理论和强化学习都习惯把“当前可用于决策的信息”收进状态，把“智能体施加的干预”写成动作，把“任务好坏”写成奖励。第三层来自工程实践。真实世界太复杂，研究者需要选出一组能训练、能预测、能服务决策的接口变量，再让神经网络学习这些变量之间的关系。

因此， $o_t, a_t, r_t, d_t, s_t, z_t, h_t$ 这组记号有明确的信息分工： o_t 承接传感器输入， a_t 承接干预， r_t 承接目标， d_t 承接结束事件， s_t 或 z_t, h_t 承接内部状态。这个抽象无法把世界全部装进去，它追求的是“对预测和控制足够充分”。例如开车时，车道线、前车距离、车速和红绿灯状态足以支撑多数驾驶动作；路边广告牌字体对控制价值很弱，模型可以把它弱化。世界模型的核心难题就在这里：压缩掉无关细节，同时保留会改变未来和奖励的信息。

在一个常见的时间步 t ，常见记号如下：

记号	含义
o_t	第 t 个时间步的观测，常见形式包括图像、视频帧、点云、文本描述或多模态输入。
a_t	第 t 个时间步执行的动作，可以是离散动作，也可以是连续控制量。
r_t	第 t 个时间步得到的奖励，用来衡量动作对任务目标的影响。
d_t	第 t 个时间步的终止信号，表示当前回合是否结束。
s_t	环境状态，通常指理论上的完整状态。很多任务里它不可直接观测。
z_t	潜变量或 latent state，用来压缩表示环境信息。
h_t	确定性隐藏状态，常见于 RNN 或 RSSM，用来保存历史信息。
ϕ	编码器、推断网络或表征网络的参数。
θ	动力学模型、解码器或策略模型的参数。
T	一个轨迹片的长度。

从符号上看，世界模型最常见的任务形式可以写成：

$$p_{\theta}(o_{1:T}, r_{1:T}, d_{1:T} \mid a_{1:T}). \quad (1)$$

式中, $o_{1:T}$ 表示从第 1 步到第 T 步的观测序列, $r_{1:T}$ 表示对应的奖励序列, $d_{1:T}$ 表示终止信号序列, $a_{1:T}$ 表示动作序列, p_θ 表示由参数 θ 控制的条件分布。竖线 $|$ 表示“在动作序列给定的条件下”。这个式子表达的意思很直接: 模型希望根据动作历史, 预测未来观测、奖励和终止情况。这里每个变量都有存在理由。 $a_{1:T}$ 把“智能体做了什么”放进模型; 去掉动作以后, 模型只能学到平均未来, 例如同一辆车在路口可能左转、右转、直行, 缺少方向盘动作时模型会把这些未来混在一起。 $r_{1:T}$ 让模型知道哪些未来对任务有价值; 缺少奖励以后, 模型可能生成很真实的画面, 却难以告诉策略哪条路更值得走。 $d_{1:T}$ 负责表达结束事件, 例如游戏失败、机器人摔倒或自动驾驶仿真终止; 删掉它以后, 模型会倾向于继续往后推演, 策略也更难学会规避危险结局。 T 控制模型一次看多远, 太短会缺少长期后果, 太长会放大模型误差。

一个生活化例子是外卖骑手规划路线。 $a_{1:T}$ 是一串动作: 直行、左转、等待红灯; $o_{1:T}$ 是沿途看到的路况; $r_{1:T}$ 可以表示是否按时送达、是否绕远; $d_{1:T}$ 表示订单完成或路线失败。世界模型要学的核心, 在于这些动作会把骑手带到怎样的未来; 某一帧街景的纹理只是背景信息。

如果任务更偏向控制, 模型还会显式学习状态转移, 例如:

$$\hat{s}_{t+1}, \hat{r}_{t+1}, \hat{o}_{t+1} = f_\theta(s_t, a_t). \quad (2)$$

式中, $\hat{s}_{t+1}$ 、 $\hat{r}_{t+1}$ 、 $\hat{o}_{t+1}$ 分别表示模型对下一时刻状态、奖励和观测的预测值, s_t 表示当前状态, a_t 表示当前动作, f_θ 表示由参数 θ 控制的动力学函数。这个公式强调的是一步预测。在真正的算法里, 模型还会沿着时间轴反复滚动, 形成多步预测或规划。这里的 s_t 像是模型内部的“摘要”。如果直接把所有图像像素当状态, 计算会很重, 模型还会被光照、纹理、背景这类细节牵着走; 如果状态摘要过小, 又会丢掉速度、位置、接触关系等关键信息。 a_t 是动作条件, 缺少它就无法区分“踩刹车后车速下降”和“踩油门后车速上升”。 f_θ 承担的本质工作, 是学习动作怎样改变状态。

一个简明计算例子: 假设扫地机器人当前状态 s_t 记录为“电量 80%、位置在客厅、前方无阻碍”, 动作 a_t 是“前进 1 米”。模型可能输出 $\hat{s}_{t+1}$ 为“电量 79.8%、位置接近沙发”, $\hat{o}_{t+1}$ 为“前方出现桌腿”, $\hat{r}_{t+1}$ 为 +1, 表示清扫覆盖面积增加。这个例子里, 状态、动作、观测和奖励各自承担不同功能, 合在一起才构成可用于控制的预测。

2 从 Dyna 到深度模型

2.1 Dyna 的思想

Sutton 提出的 Dyna 架构把三件事连在一起: 真实环境交互、模型学习、基于模型的规划 [1]。它的基本直觉很朴素, 智能体先从真实环境里学到一点规律, 再把这些规律拿到内部反复练习。这样一来, 真实交互的样本就能被更充分地利用。

Dyna 里的“内部反复练习”通常有很具体的形式。智能体先用真实交互样本学习一个近似环境模型, 例如保存或拟合 $(s_t, a_t) \mapsto (s_{t+1}, r_{t+1})$ 。随后从已经访问过的状态、动作或 replay buffer 中抽取起点, 让模型生成模拟转移, 再用这些模拟转移更新价值函数或策略。表格式 Dyna 中, 这一步常表现为若干次 planning backup; 深度世界模型中, 则表现为在 learned model 里 rollout 多条短轨迹, 并用预测到的奖励、终止信号和 latent state 更新 actor、critic 或规划器。

可以把 Dyna 想成通勤路线学习。第一次走新路线时, 只能边走边试; 走过几次后, 脑中会形成“从东门出发时哪个路口容易堵”的路线模型。基于这个路线模型, 可以离线比较几条可能

路径，降低下一次真实尝试的成本。Dyna 的模型学习对应“形成路线印象”，基于模型的规划对应“在内部试走几条路线”。

早期 Dyna 主要面对低维状态表，现代世界模型面对的是高维图像、长时序语义和复杂动作空间。这一变化带来的难点很明确：

- 观测维度大。
- 未来可能多解。
- 长时序误差会累积。
- 控制目标和视觉重建目标之间可能存在张力。

站在 2026 年回看，Dyna 的意义主要在思想层面。它没有解决视觉表征、长时序稳定性和神经网络训练问题，但它给出了“真实交互 + 内部模型 + 规划更新”的基本闭环。后来的 Dreamer、MuZero、TD-MPC 都可以看作这条闭环在深度学习时代的不同实现。

2.2 MBRL 的一个长期问题

模型越往前滚，误差越容易积累。当模型被用于规划时，策略还可能学会利用模型漏洞，找到在模型里看起来很优、在真实环境里却很差的动作。这种偏差常见于四个来源。第一，模型见过的数据覆盖有限，策略在规划时会搜索到训练数据外的动作序列，模型在这些区域缺少校准。第二，抽象状态遗漏了关键因素，例如机器人抓取任务里少记录了摩擦、接触角度或物体质量，短期预测看起来合理，真正执行时会失败。第三，损失函数和控制目标有错位，模型可能很会重建背景纹理，却没有学准奖励相关的接触事件。第四，多步 rollout 会放大小误差，第一步位置偏差 1 厘米，滚动十几步后可能变成完全不同的状态。

Rollout 指从一个起始状态出发，反复使用同一个模型向前预测。一步 rollout 只预测下一状态和奖励；多步 rollout 会把上一步预测出的状态再作为下一步输入，继续预测后续状态。若写成公式，可以表示为：

$$\hat{s}_{t+k+1}, \hat{r}_{t+k+1} = f_{\theta}(\hat{s}_{t+k}, a_{t+k}), \quad k = 0, \dots, H-1. \quad (3)$$

这里 H 是 rollout horizon。短 rollout 误差较小，但能看到的后果有限；长 rollout 能服务更远期规划，同时会把模型误差逐步放大。

可以把这种现象想成看地图找近路。地图上某条小路看起来连通，真实世界里可能有围栏或施工。规划器会选择地图里的最短路线，真正走过去才发现无法通行。问题未必来自地图整体很差，常常来自某个对决策极关键的细节缺失。世界模型也是如此：平均预测误差很低，仍可能在策略最爱利用的区域出错。因此，很多后续工作都转向两个方向：

- 只在短时间尺度里信任模型。
- 让模型学习对控制有用的抽象，降低对全量重建的依赖。

SimPLe 把视频预测式模型用于 Atari [5]，MBPO 强调短 rollout 的实用价值 [6]。这两篇工作共同指向一个经验：世界模型的价值取决于它是否降低控制难度；精确复原世界只是其中一种

训练目标。二者的关系可以理解为“外层目标”和“训练手段”。外层目标是让智能体更容易选择好动作，例如更快知道哪条轨迹会带来高奖励、哪种状态危险、哪段未来值得继续搜索。精确复原世界是一类常用手段，因为好的视觉重建通常说明模型保留了大量环境信息。可是控制任务关心的是决策相关信息。自动驾驶模型可以弱化天空云层的形状，但必须保留前车刹车灯和车道边界；机器人推箱子模型可以弱化地板纹理，但必须保留箱子位置、接触关系和摩擦趋势。

所以，评价世界模型时要同时看两类信号。重建误差、预测误差能检查模型是否理解观测分布；reward、success rate、planning return 能检查模型是否真的降低了控制难度。若重建很好、控制很差，说明模型可能把容量花在了任务弱相关的细节上；若控制很好、重建一般，说明模型学到的抽象已经足以服务决策。

3 VAE: 世界模型里最常见的底层积木

理解现代世界模型时，VAE 是绕不开的基础。它在视觉世界模型里常被用来把像素压缩成更紧凑的潜变量，再把动力学学习放到潜空间里完成。

3.1 为什么需要 VAE

普通自编码器只关心压缩和重建。VAE 在这个基础上再加入概率结构和先验约束，让潜空间更平滑，也更容易采样。对世界模型来说，这个特性很重要，因为环境未来常常存在多种合理走向。如果 latent space 杂乱无章，后续的动力学模型就很难在里面滚动。

潜变量可以看作对世界的一种抽象映射。原始观测 x 可能是整张图像、视频片段或多模态输入，里面混合了对象位置、速度、光照、纹理、遮挡、任务目标等因素。编码器把 x 映射到 z ，目标是把“对解释当前观测和预测未来有用的因素”压进更低维的内部坐标。这个映射由重建、KL 约束、动力学预测和下游控制目标共同塑形，不依赖人工预先规定的语义标签。

普通自编码器可以写成两步：编码器 E_ϕ 把输入 x 压成 $z = E_\phi(x)$ ，解码器 D_θ 再从 z 重建 $\hat{x} = D_\theta(z)$ 。压缩的意义是迫使模型把输入里最有解释力的信息放进瓶颈变量 z ，重建的意义是检查这些信息是否足以恢复原始样本。用图像例子说，一张 $64 \times 64 \times 3$ 的图片有 12288 个像素值，若把它压成 32 维向量，模型就必须舍弃很多细节，只保留足以解释主体、位置、颜色和运动线索的信息。

信息丢失确实会发生，关键在于让丢失集中到任务弱相关细节上。常见办法包括：调节瓶颈维度，让 z 容量足够承载关键因素；使用重建损失，让模型保留能解释输入的内容；加入奖励预测、终止预测或动作条件预测，让模型保留会影响控制的因素；用下游任务表现检验表示质量。若模型重建出了漂亮背景，却丢掉物体速度，世界模型在控制上会失败；若模型重建稍粗糙，却保留位置、速度和接触关系，它仍然可能是更好的控制表示。

重建方向由训练目标约束。自编码器训练时，解码器输出 $\hat{x}$ 会和原始输入 x 比较，常见损失包括均方误差、交叉熵或像素分布的负对数似然。这个比较会持续告诉模型哪些像素、边缘、颜色和结构需要被恢复。若加入奖励预测或动力学预测，模型还会收到“哪些信息影响未来和任务”的额外信号。对世界模型而言，重建损失负责守住观测层面的合理性，任务相关损失负责提醒模型保留控制所需因素。

潜空间可以理解为输入经过编码后落入的内部坐标系。一张车道图像、一段机器人观测或一个 Atari 画面送进编码器后，输出的 z 就是这份观测在潜空间中的位置。说潜空间“平滑”，指

相近的输入在潜空间里也相近，沿着潜空间缓慢移动会对应语义上连续的变化。例如小车位置从左到右平移， z 也应该沿着某个方向连续变化。说潜空间“容易采样”，指从先验分布取一个 z ，解码后有较大概率得到合理样本，避免落到大片无意义区域。

为什么一定要设置 z ？因为原始图像里包含大量和控制关系较弱的细节，例如墙面纹理、光照变化、远处背景。 z 的作用是把观测压成一个“够用的内部摘要”。如果删掉 z ，模型就要直接处理像素级未来；这会让训练成本和误差积累明显上升。如果把 z 换成一个普通确定性向量，模型又很难表达多种合理未来，例如同一张路口图像后面可能出现行人、车辆或空路面。

3.2 VAE 的核心公式

先解释“后验”。在概率模型里，先验 $p(z)$ 表示看见样本前对潜变量的默认相信程度；后验 $p_\theta(z | x)$ 表示看见样本 x 后，对潜变量 z 的更新判断。直觉上，如果 x 是一张“向左转弯的道路图像”，后验就应该把 z 推到能解释“车道方向、道路边界、车辆姿态”的区域。

真实后验 $p_\theta(z | x)$ 往往很难直接算，因为它需要对所有可能的 z 做归一化积分：

$$p_\theta(z | x) = \frac{p_\theta(x, z)}{p_\theta(x)} = \frac{p_\theta(x | z)p(z)}{\int p_\theta(x | z)p(z) dz}. \quad (4)$$

分母里的积分会遍历整个潜空间。高维 z 下，这个积分通常没有解析解，网格积分也会随维度指数级变贵。VAE 用一个神经网络 $q_\phi(z | x)$ 去近似真实后验。这个网络的输入是样本 x ，输出是分布参数，例如均值 $\mu_\phi(x)$ 和标准差 $\sigma_\phi(x)$ 。这样一来，推断过程就从“每次都求高维积分”变成“学习一个从 x 到后验参数的函数”。

神经网络近似积分的核心在于学习可用的后验参数映射，绕开逐项计算积分的路线。更准确地说，神经网络作为通用函数逼近器，可以学习一个近似映射：

$$x \mapsto (\mu_\phi(x), \sigma_\phi(x)). \quad (5)$$

训练时，ELBO 通过重建项和 KL 项给出可优化信号，促使这个映射输出的分布接近有用的后验。期望项通常用 Monte Carlo 采样估计，KL 项在对角高斯设定下有闭式解。因此，VAE 把难以直接求解的贝叶斯后验推断，改写成了可微分的函数学习问题。

VAE 常用的近似后验可写成：

$$q_\phi(z | x) = \mathcal{N}(z; \mu_\phi(x), \text{diag}(\sigma_\phi^2(x))). \quad (6)$$

式中， x 表示输入样本，常见形式是图像或图像片段； z 表示潜变量； $q_\phi(z | x)$ 表示编码器给出的近似后验分布； ϕ 表示编码器参数； $\mu_\phi(x)$ 表示均值向量； $\sigma_\phi(x)$ 表示标准差向量； $\mathcal{N}(\cdot)$ 表示高斯分布； $\text{diag}(\sigma_\phi^2(x))$ 表示以方差向量为对角线的协方差矩阵。这条公式的本质，是让编码器输出一个带有不确定范围的分布区域，区别于单个确定点。 $\mu_\phi(x)$ 给出区域中心， $\sigma_\phi(x)$ 给出不确定范围。若只保留 $\mu_\phi(x)$ ，同一类图像会被压到很硬的点上，后续采样容易断裂；若删掉 $\sigma_\phi(x)$ ，模型就失去了表达不确定性的通道。对世界模型来说，不确定性很有价值，因为未来天然带有分支。

这里常用高斯分布，主要有三个原因。第一，高斯分布由均值和方差刻画，编码器输出简单，训练稳定。第二，对角高斯与标准高斯之间的 KL 散度有闭式公式，避免每次训练都做昂贵数值积分。第三，重参数化形式简单，便于梯度回传。高斯只是常见默认选择；潜变量也可以采

用 Bernoulli、categorical、混合分布、normalizing flow 或离散 codebook。分布越灵活，表达能力通常越强，训练、采样和 KL 估计也会更复杂。世界模型工程中经常先用对角高斯，是因为它在表达能力、稳定性和实现成本之间取得了较好的平衡。

输入输出结构可以具体想成这样：输入 x 是一张 $64 \times 64 \times 3$ 图像，编码器经过卷积和全连接层后输出两组长度为 2 的向量，例如 $\mu_\phi(x) = (1.0, -0.5)$ ， $\sigma_\phi(x) = (0.2, 0.5)$ 。这时近似后验就是一个二维高斯分布，第一维大多落在 1.0 附近，第二维大多落在 -0.5 附近。随后模型从这个分布里采样 z ，再交给解码器或动力学模型。

VAE 的训练目标常写成：

$$\mathcal{L}_{\text{VAE}}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)). \quad (7)$$

式中， $p_\theta(x|z)$ 表示解码器给出的重建分布， θ 表示解码器参数， $p(z)$ 表示先验分布，通常取标准高斯 $\mathcal{N}(0, I)$ ， $\text{KL}(\cdot \| \cdot)$ 表示 Kullback-Leibler 散度， $\mathbb{E}_{q_\phi(z|x)}[\cdot]$ 表示对近似后验采样后的期望。其中， $\mathcal{L}_{\text{VAE}}(x)$ 表示单个样本 x 对应的 VAE 训练目标， $\log$ 表示自然对数， $\|$ 左右两侧分别是 KL 散度比较的两个分布， z 和 ϕ 沿用上一条公式中的含义。这一目标由两部分组成，第一部分推动重建，第二部分把潜空间拉回到可采样、可组织的形状。如果去掉重建项， z 可能变得很规整，却没有保留输入信息；如果去掉 KL 项，模型会得到一个好用但杂乱的编码空间，采样时容易落到训练数据没有覆盖的地方。VAE 的训练目标本质上是在做平衡：让 z 既能解释输入，又能形成平滑、可采样的空间。

KL 项比较的是 $q_\phi(z|x)$ 和 $p(z)$ 。前者来自具体输入，负责解释当前样本；后者是统一先验，负责给整个潜空间定形。这样取的原因很实际：训练时每个样本都有自己的近似后验，生成或想象时模型常常需要从统一先验附近出发。若这些后验分布分散在彼此隔开的孤岛上，采样时很容易落到空洞区域。KL 项会把这些岛拉向同一个可控坐标系，让模型在训练和采样之间少一些断层。

对角高斯到标准高斯的 KL 可以写成：

$$\text{KL}(\mathcal{N}(\mu, \text{diag}(\sigma^2)) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1). \quad (8)$$

式中， i 表示潜变量的维度索引， μ_i 和 σ_i 表示第 i 维的均值和标准差。假设 z 有两维， $\mu = (1.0, -0.5)$ ， $\sigma = (0.5, 1.5)$ 。第一维贡献为 $1.0^2 + 0.5^2 - \log(0.5^2) - 1 = 1 + 0.25 - \log 0.25 - 1 \approx 1.636$ ；第二维贡献为 $(-0.5)^2 + 1.5^2 - \log(1.5^2) - 1 = 0.25 + 2.25 - \log 2.25 - 1 \approx 0.689$ 。乘上 $\frac{1}{2}$ 后，KL 约为 1.1625。这个数越大，说明近似后验离标准高斯先验越远，潜空间约束越强。

一个极简计算例子：假设某张图像的重建对数概率为 -18，KL 散度为 4，那么这张图像对应的目标值就是 $-18 - 4 = -22$ 。如果另一个编码方式重建更好，重建对数概率达到 -12，但 KL 散度升到 15，目标值就是 -27，反而更低。这个例子说明，VAE 不会只奖励重建清晰度，也会惩罚潜空间过于散乱。

训练时常重用参数化技巧：

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (9)$$

式中， ϵ 表示从标准高斯采样得到的噪声， $\mathcal{N}$ 表示高斯分布， 0 表示零均值向量， I 表示单位矩阵， $\odot$ 表示按元素相乘， $\mu_\phi(x)$ 和 $\sigma_\phi(x)$ 沿用前文含义，分别对应编码器输出的均值向量和标准

差向量。这种写法把随机采样拆成“可导的均值与方差”加上“外部噪声”，这样梯度就能顺利回传到编码器。直接从 $q_\phi(z|x)$ 采样会把随机节点放在计算图中间，梯度很难告诉编码器“均值和方差该怎样改”。重参数化把随机性移到 ϵ 上， z 对 $\mu_\phi(x)$ 和 $\sigma_\phi(x)$ 仍是普通可导表达式。例如第一维 $z_1 = \mu_1 + \sigma_1 \epsilon_1$ ，若损失希望 z_1 更大，梯度可以直接推动 μ_1 增大，或在合适情况下调整 σ_1 。

重参数化的意义可以分成两层。数学上，它把“对分布参数求梯度的采样期望”改写为“对固定噪声变换后的确定性函数求梯度”，对应 pathwise gradient estimator。工程上，它让 VAE 可以像普通神经网络一样端到端训练：编码器输出均值和方差，采样得到 z ，解码器重建 x ，损失再一路反传到编码器。没有这一步，VAE 的随机采样会让训练信号难以稳定传回 $q_\phi(z|x)$ 。

一个二维计算例子：假设编码器给出 $\mu_\phi(x) = (1.0, -0.5)$ ， $\sigma_\phi(x) = (0.2, 0.5)$ ，这次采到的噪声是 $\epsilon = (0.3, -1.0)$ 。按元素计算后，第一维潜变量为 $1.0 + 0.2 \times 0.3 = 1.06$ ，第二维潜变量为 $-0.5 + 0.5 \times (-1.0) = -1.0$ 。所以这次采样得到 $z = (1.06, -1.0)$ 。这个例子说明， μ 决定大致位置， σ 决定扰动幅度， ϵ 提供随机性。

3.3 VAE 在世界模型中的作用

当输入是高维图像时，直接在像素空间做动力学预测成本高、难度大，也容易被细枝末节干扰。VAE 的作用是把原始观测压成一个更紧凑的 latent code，让后续动力学模型在这个空间里预测未来。这一步有三个直接收益：

- 降低维度。
- 弱化局部噪声。
- 让多步预测更可行。

4 Ha 和 Schmidhuber 的 World Models

World Models 是这一研究脉络的标志性工作 [3]。它把系统拆成三部分：

- **Vision model:** 用 VAE 把图像压缩成潜变量。
- **Memory model:** 用带有混合密度输出的 RNN 学习时间演化。
- **Controller:** 在潜空间和记忆状态上输出动作。

这里的关键点有两个。第一，视觉模块承担的是压缩任务，它把高维图像变成低维 latent state。第二，记忆模块面对的是多种未来可能性，因此使用混合密度输出会更合适，因为同一个当前状态和动作可能对应多个合理未来。

混合密度输出可以理解为“同时给出几种可能未来及其概率”。若记忆模块要预测下一步 latent z_{t+1} ，它可以输出：

$$p(z_{t+1} | h_t, a_t) = \sum_{k=1}^K \pi_k \mathcal{N}(z_{t+1}; \mu_k, \text{diag}(\sigma_k^2)). \quad (10)$$

式中, K 表示混合成分数量, π_k 表示第 k 个未来分支的权重, 满足 $\sum_k \pi_k = 1$; μ_k 表示第 k 个高斯分布的中心; σ_k 表示第 k 个高斯分布的标准差; h_t 是 RNN 记忆状态, a_t 是当前动作。若 z_{t+1} 是一维变量, $K = 3$, 模型可能输出 $\pi = (0.6, 0.3, 0.1)$, $\mu = (-1.0, 0.4, 2.0)$, $\sigma = (0.2, 0.3, 0.5)$ 。这表示下一步最可能落在 -1.0 附近, 也有一定概率落在 0.4 附近, 小概率跳到 2.0 附近。

混合输出的意义在于避免把多种未来平均成模糊未来。例如游戏角色站在岔路口, 向前走后可能进入左房间, 也可能进入右房间。单一高斯均值可能落在两个房间中间, 生成一个现实中不存在的状态; 混合密度可以把左房间和右房间当成两个分支分别表示。若 z 有 D 维, 模型通常会输出 K 个权重、 $K \times D$ 个均值和 $K \times D$ 个标准差, 形成一个可采样的多峰预测分布。

这篇工作的重要贡献体现在一条清晰路线: 先学习内部世界, 再在内部世界里训练控制器。对第一次接触世界模型的人来说, 这篇论文非常适合作为第一篇精读对象。

站在 2026 年回看, 这篇论文的价值主要体现在范式清晰, 性能天花板属于次要因素。它把图像压缩、时间预测和控制器三件事拆开, 让后来的论文有了一个容易讨论的模板。阅读重点应放在这套分工上: 视觉模块负责把世界变成内部状态, 记忆模块负责推演内部状态, 控制器负责在内部状态上行动。

一个贴近日常的例子是学开车。刚开始看到的是整幅道路画面, 里面有树、广告牌、行人、车道线。真正影响驾驶的摘要只有一部分: 前车距离、车道位置、红绿灯、旁边车辆速度。Vision model 对应“提取驾驶相关摘要”, Memory model 对应“根据过去几秒判断车流走势”, Controller 对应“决定刹车、转向或加速”。如果视觉摘要里保留了广告牌颜色, 却丢掉前车距离, 控制器会很危险; 如果记忆模块只看当前帧, 遇到突然变道的车也很难提前反应。

5 Planet、Dreamer 与 latent dynamics

5.1 RSSM 是什么

Planet 和 Dreamer 系列最重要的共同基础, 是 recurrent state-space model, 简称 RSSM [7]。它可以写成:

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}). \quad (11)$$

式中, h_t 表示第 t 个时间步的确定性隐藏状态, h_{t-1} 表示上一个时间步的隐藏状态, z_{t-1} 表示上一个时间步的随机潜变量, a_{t-1} 表示上一个时间步执行的动作, f_θ 表示带参数 θ 的递归更新函数。这个隐藏状态会把历史信息压缩到一个向量里, 便于后续预测。为什么 RSSM 同时需要 h_t 和 z_t ? h_t 像一本连续更新的笔记, 负责保存历史; z_t 像当前时刻的随机解释, 负责承接不确定性。只用 h_t 会让模型偏向确定性预测, 多峰未来容易被平均; 只用 z_t 会让长期记忆变弱, 模型难以记住“刚才发生过什么”。动作 a_{t-1} 也必须进入更新函数, 因为同一个历史状态在不同动作后会走向不同未来。

厨房烧水可以作为类比。 h_t 记录过去几分钟的信息: 水壶已经开火三分钟、厨房门开着、火力刚被调小; z_t 表达当前状态里的不确定性: 水快开了还是刚冒泡; a_t 是动作: 继续加热、关火或拿走水壶。若删掉历史记忆, 只看当前一眼, 模型很难判断水温趋势; 若删掉随机潜变量, 它会把“快开”和“刚冒泡”混成一个模糊判断。

一个一维玩具计算可以展示更新方式。假设临时把 f_θ 写成 $0.5h_{t-1} + 0.3z_{t-1} + 0.2a_{t-1}$, 并

给定 $h_{t-1} = 0.4$, $z_{t-1} = 0.6$, $a_{t-1} = -1$, 则 $h_t = 0.5 \times 0.4 + 0.3 \times 0.6 + 0.2 \times (-1) = 0.18$ 。真实 RSSM 会用神经网络做这个更新, 玩具式子只是展示三类输入怎样共同改变记忆状态。

RSSM 中常见的随机部分还会写成:

$$q_\phi(z_t | h_t, o_t), \quad p_\theta(z_t | h_t). \quad (12)$$

式中, $q_\phi(z_t | h_t, o_t)$ 表示训练阶段使用的后验分布, 它会同时看隐藏状态和当前观测; $p_\theta(z_t | h_t)$ 表示先验分布, 它只看隐藏状态; z_t 表示当前潜变量; o_t 表示当前观测; ϕ 表示后验推断网络的参数; θ 表示先验动力学网络的参数。训练时, 后验帮助模型吸收真实观测信息; 想象 rollout 时, 先验负责往前生成未来。这对先验和后验很关键。后验 q_ϕ 像考试后的订正, 可以看见真实观测 o_t , 因此能把模型拉回现实; 先验 p_θ 像闭眼推演, 只能根据历史 h_t 往前猜。训练阶段若缺少后验, 模型很难从真实图像中校准内部状态; 想象阶段若依赖后验, 模型又会离不开真实观测, 无法独立 rollout。PlaNet 和 Dreamer 的核心技巧之一, 就是让这两者在训练时对齐。

例如, 先验可能根据历史判断 z_t 大约服从均值 0、标准差 1 的高斯; 真实观测进来后, 后验把均值修正到 0.8、标准差缩小到 0.2。这说明观测让模型更确定地定位了当前状态。训练时让先验逐渐接近后验, 模型在没有真实观测的想象阶段才更可靠。

前面的递归更新公式和这组先验/后验公式是一前一后的关系。 $h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1})$ 先把过去信息推进到当前时刻, 得到一份确定性记忆; 随后 $q_\phi(z_t | h_t, o_t)$ 或 $p_\theta(z_t | h_t)$ 再为当前时刻选择随机潜变量 z_t 。最终模型真正拿来预测奖励、终止信号和图像重建的状态, 通常是 (h_t, z_t) 的组合。

h_t 和 z_t 的分工可以这样记。 h_t 负责“我一路走到这里的历史”, 例如上一秒速度、刚才做过的动作、之前看到的目标; z_t 负责“当前这一刻还有哪些不确定解释”, 例如遮挡后面是否有障碍、下一帧球会从左边还是右边出现。RSSM 同时保留两者, 是为了兼顾长时记忆和局部随机性。

从输入输出角度看, RSSM 训练时通常有三条路径。第一条是 representation path: 输入历史隐藏状态 h_t 和真实观测 o_t , 得到后验 $q_\phi(z_t | h_t, o_t)$ 。第二条是 dynamics path: 只输入历史隐藏状态 h_t , 得到先验 $p_\theta(z_t | h_t)$ 。第三条是 prediction path: 把 (h_t, z_t) 送入观测解码器、奖励头和继续信号头, 预测 $\hat{o}_t, \hat{r}_t, \hat{c}_t$ 。训练时后验保证当前状态吸收真实观测, 先验被 KL 项拉近后验; 想象时真实观测缺席, 只能沿先验递推。这个结构解释了 RSSM 为什么既能利用真实数据校准, 又能在模型内部独立 rollout。

5.2 PlaNet 的位置

PlaNet 先在像素输入上学习 latent dynamics, 再在 latent space 里做规划 [7]。它延续了“压缩后再预测”的思想, 但把重点放在规划过程本身。这一思路的价值很高, 因为它把高维视觉问题改写成了更紧凑的状态转移问题。

PlaNet 值得多看, 因为它把问题形式真正切到了 latent dynamics 上。它关注压缩状态空间能否支持规划, 漂亮视频生成只属于次要目标。用生活例子说, PlaNet 像是在陌生商场里找出口: 需要记住电梯、扶梯、出口标识和转角关系, 无需在脑中渲染每块地砖的纹理。规划需要的是这种结构摘要。

5.3 Dreamer 的位置

Dreamer 进一步把训练方式改成“在模型里想象未来，再用想象轨迹更新策略” [8]。它的流程很清楚：

1. 用真实交互数据训练世界模型。
2. 从当前 latent state 出发，在模型里滚动出 imagined trajectory。
3. 用这些轨迹训练 actor 和 critic。

这种做法的优点很直接，真实环境交互会更省，策略优化会更依赖内部模拟。DreamerV2 用离散 latent 表示增强了稳定性 [9]。DreamerV3 则进一步强调跨任务通用性，覆盖了连续控制、Atari 和 Minecraft 等不同环境 [10]。

Dreamer 是世界模型主线中最值得投入时间的部分之一。它的本质在于把策略学习搬进模型内部，生成模型只是支撑手段之一。真实环境给世界模型提供校准数据；世界模型再批量生成想象轨迹；actor 和 critic 在这些轨迹上学习。这样一来，智能体可以把有限真实经验放大成大量内部练习。

举个贴近生活的例子：学滑板的人不会每次都真的冲下坡去试错。看过几次摔倒后，他会在脑中预演“如果重心太靠后会怎样”“如果提前转肩会怎样”。Dreamer 的 imagined trajectory 就像这种脑内练习。它当然会出错，所以世界模型必须持续用真实交互修正；当脑内练习足够接近现实时，学习效率会显著提高。

DreamerV2 的离散 latent 表示也值得理解。连续向量像在一张平滑地图上移动，离散表示像把地图切成可命名的格子。对于 Atari、Minecraft 这类场景，离散内部状态有时更接近“事件”和“对象”的结构，例如吃到道具、进入房间、触发危险。DreamerV3 的意义则在于把这条路线从单个环境推进到更广泛的任务集合。站在 2026 年看，Dreamer 系列仍是理解 latent world model RL 的核心路线。

5.4 PlaNet 到 DreamerV3 的详细对比

工作	主要目标	决策方式	阅读重点
PlaNet	从像素学习 RSSM，并在 latent space 里规划。	每次决策时用模型预测多条候选动作序列，再选回报较高的一条。	重点看 latent dynamics 怎样把高维视觉控制变成低维规划问题。
Dreamer	把策略学习搬进 latent imagination。	用世界模型生成想象轨迹，在这些轨迹上训练 actor 和 critic。	重点看真实经验、世界模型、想象轨迹、策略更新怎样形成闭环。
DreamerV2	用离散 latent 提升 Atari 等任务中的稳定性。	继续使用 latent imagination 训练策略。	重点看离散表示如何表达事件、对象和游戏状态。

DreamerV3	追求跨领域通用性，覆盖连续控制、Atari、Minecraft 等环境。	用更统一的训练配置和尺度处理，让同一框架迁移到更多任务。	重点看通用化工程：损失尺度、奖励尺度、模型容量和固定超参数。
-----------	--------------------------------------	------------------------------	--------------------------------

这个表把四篇论文连成一条线。PlaNet 解决的是“能否在潜空间里规划”；Dreamer 解决的是“能否把规划经验变成可训练策略”；DreamerV2 解决的是“离散事件丰富的环境里 latent 怎样更稳”；DreamerV3 解决的是“同一套方法怎样跨任务工作”。从学习顺序上看，PlaNet 是桥，Dreamer 是主干，DreamerV2 和 DreamerV3 是主干上的增强。

5.5 latent space 与 latent imagination

latent space 和 latent imagination 很容易混在一起，但二者指向不同层面。

latent space 是表示空间。它回答“观测被压缩到哪里”，例如一帧图像经过编码器后得到 z_t ，或者 RSSM 得到 (h_t, z_t) 。这个空间的质量取决于编码器、先验约束、重建损失、奖励预测和动力学预测。好的 latent space 应当把决策相关因素放近，把无关纹理和噪声弱化。

latent imagination 是在这个表示空间里向前推演。它回答“从当前 latent state 出发，如果执行一串动作，未来 latent state、奖励和终止信号会怎样变化”。Dreamer 的想象轨迹可以写成：

$$(h_{t+k+1}, z_{t+k+1}) \sim p_\theta(\cdot | h_{t+k}, z_{t+k}, a_{t+k}), \quad k = 0, \dots, H-1. \quad (13)$$

因此，latent space 是舞台，latent imagination 是在舞台上展开的内部 rollout。前者强调状态如何编码，后者强调模型如何利用这些状态生成可训练的未来经验。

5.6 Dreamer 的训练闭环

Dreamer 的世界模型训练通常包含观测重建、奖励预测、终止或继续信号预测，以及先验和后验的对齐。可以简化写成：

$$\begin{aligned} \mathcal{L}_{\text{model}} = \sum_t [& -\log p_\theta(o_t | h_t, z_t) - \log p_\theta(r_t | h_t, z_t) - \log p_\theta(c_t | h_t, z_t) \\ & + \beta \text{KL}(q_\phi(z_t | h_t, o_t) \| p_\theta(z_t | h_t))]. \end{aligned} \quad (14)$$

式中， $\mathcal{L}_{\text{model}}$ 表示世界模型损失； o_t 是第 t 步观测； r_t 是奖励； c_t 是 continue 信号，表示轨迹是否继续； h_t 是确定性隐藏状态； z_t 是随机潜变量； $p_\theta(o_t | h_t, z_t)$ 、 $p_\theta(r_t | h_t, z_t)$ 、 $p_\theta(c_t | h_t, z_t)$ 分别是观测、奖励和继续信号的预测分布； q_ϕ 是看见当前观测后的后验； $p_\theta(z_t | h_t)$ 是只依赖历史的先验； β 控制 KL 对齐项的权重。

这个损失的直觉很清楚。观测重建让 latent state 保留画面信息；奖励预测让 latent state 保留任务价值；continue 预测让模型知道何时会失败或结束；KL 对齐让训练时的后验和想象时的先验靠近。若奖励预测很差，actor 在想象中会追逐错误目标；若 continue 预测很差，模型会把已经失败的轨迹继续往后推；若先验和后验差距大，模型在真实数据上看起来正常，进入想象 rollout 后会迅速漂移。

模型学好后, Dreamer 从 replay buffer 里取一批真实轨迹, 把它们编码成 latent state, 再让 actor 在世界模型里连续选择动作:

$$a_{t+k} \sim \pi_\psi(a \mid h_{t+k}, z_{t+k}), \quad z_{t+k+1} \sim p_\theta(z \mid h_{t+k+1}). \quad (15)$$

式中, π_ψ 表示 actor 策略, ψ 是 actor 参数, a_{t+k} 是想象轨迹中第 k 步动作, z_{t+k+1} 是模型根据先验采样出的下一步随机潜变量。这里的关键在于, actor 训练时减少对真实环境逐步访问的依赖, 改为在世界模型里批量练习。

critic 负责评估 latent state 的长期价值, actor 负责选择让价值更高的动作。一个简化的想象回报可以写成:

$$\hat{G}_t = \sum_{k=0}^{H-1} \gamma^k \hat{r}_{t+k+1} + \gamma^H V_\xi(h_{t+H}, z_{t+H}). \quad (16)$$

式中, $\hat{G}_t$ 表示从当前 latent state 出发的想象回报; H 是想象长度; γ 是折扣因子; $\hat{r}_{t+k+1}$ 是世界模型预测的奖励; V_ξ 是 critic, ξ 是 critic 参数; 最后一项表示想象到第 H 步后, 用 critic 估计更远未来。举例来说, 若 $H = 3$ 、 $\gamma = 0.9$, 模型预测奖励为 $1, 2, 2$, critic 估计终点价值为 5 , 则 $\hat{G}_t = 1 + 0.9 \times 2 + 0.9^2 \times 2 + 0.9^3 \times 5 = 8.065$ 。actor 会被训练成更倾向产生这种高回报想象轨迹的动作。

critic 的训练目标是让价值估计接近想象回报, 常见形式可以简化为:

$$\mathcal{L}_{\text{critic}}(\xi) = \sum_t \left(V_\xi(h_t, z_t) - \text{sg}[\hat{G}_t] \right)^2. \quad (17)$$

这里 $\text{sg}[\hat{G}_t]$ 表示把目标回报当成常量, 避免 critic 在更新时把梯度传回目标本身。

actor 的目标是选择能提高想象回报的动作, 简化写法为:

$$\mathcal{J}_{\text{actor}}(\psi) = \mathbb{E}_{\pi_\psi, p_\theta} \left[\sum_t \hat{G}_t + \eta \mathcal{H}(\pi_\psi(\cdot \mid h_t, z_t)) \right]. \quad (18)$$

其中 $\mathcal{H}$ 是策略熵, η 控制探索强度。actor 通过世界模型产生动作, 世界模型根据动作推出未来 latent state 和奖励, critic 提供长期价值估计。三者的分工是: world model 提供可微的内部环境, critic 给状态打分, actor 学会制造高分状态序列。

从生活类比看, Dreamer 像运动员看完真实训练录像后, 在脑中大量排练动作。世界模型提供“脑内场地”, actor 是做动作的人, critic 是教练给出的长期评分。真实训练仍然重要, 因为脑内场地会漂; 想象训练也重要, 因为它可以用很低成本生成大量练习片段。Dreamer 的魅力就在于把这两者放进一个可端到端训练的系统里。

6 MuZero 与 TD-MPC

6.1 MuZero 的抽象模型

MuZero 采用的是另一条路线 [11]。它学习 representation、dynamics 和 prediction 三个模块, 在潜空间中进行搜索。这条路线的重要信号在于: 模型只需要保留对决策有用的结构。

如果一个任务的目标是博弈或搜索, 像素级重建的优先级会下降。更关键的是, 它能否支持价值估计、策略搜索和动作比较。MuZero 在围棋、国际象棋、将棋和 Atari 上都展示了这种思路的威力。

MuZero 的思想非常有穿透力。它说明世界模型无须像摄影机一样复原世界，也可以像棋手的大脑一样保留决策结构。下棋时，核心信息是棋子位置、威胁关系和后续变化，棋盘木纹没有决策价值。MuZero 学到的 hidden state 就服务这种目的：它要支持搜索、价值估计和策略改进。

如果把 MuZero 用在开车类比里，它更像一个“路线博弈器”。它重点判断某个动作序列会带来怎样的安全性和效率，挡风玻璃前的完整视频生成优先级较低。这样做的代价是解释性和真实感较弱，收益是模型更贴近决策目标。站在 2026 年看，MuZero 的长期价值在于把 world model 从“预测世界外观”推向“预测决策后果”。

6.2 TD-MPC 的控制视角

TD-MPC 把 temporal-difference learning 和 model predictive control 结合起来 [12]。它很重视短视野控制，重点在于让模型在有限预测步数内保持可靠。一个常见的规划目标可以写成：

$$a_{t:t+H-1}^* = \arg \max_{a_{t:t+H-1}} \sum_{k=0}^{H-1} \gamma^k \hat{r}_{t+k+1}. \quad (19)$$

式中， $a_{t:t+H-1}^*$ 表示从第 t 步到第 $t+H-1$ 步的一段最优动作序列， $a_{t:t+H-1}$ 表示候选动作序列， H 表示规划视野长度， γ 表示折扣因子， $\hat{r}_{t+k+1}$ 表示模型预测得到的第 $t+k+1$ 步奖励， k 表示从当前时刻开始向前计数的索引。其中， $\arg \max$ 表示选择能让目标值达到最大的一组动作， $\sum_{k=0}^{H-1}$ 表示从第 0 个预测步累加到第 $H-1$ 个预测步， γ^k 表示第 k 个预测步的折扣权重。这个目标的含义是，在有限视野里选择回报最高的动作序列。这里每个变量都有实际含义。 H 决定规划视野； H 太小，模型容易贪眼前收益， H 太大，预测误差会不断累积。 γ 决定未来奖励的权重； γ 小，系统更看重眼前， γ 接近 1，系统更看重长期。候选动作序列 $a_{t:t+H-1}$ 让规划器一次比较一段动作，避免只看单步动作。

一个简明计算例子：假设 $H = 3$ ， $\gamma = 0.9$ 。候选路线 A 的预测奖励是 3, 2, 1，折扣回报为 $3 + 0.9 \times 2 + 0.9^2 \times 1 = 5.61$ 。候选路线 B 的预测奖励是 1, 3, 3，折扣回报为 $1 + 0.9 \times 3 + 0.9^2 \times 3 = 6.13$ 。虽然 A 的第一步奖励更高，B 的三步折扣回报更高，所以规划器会选择 B。这就是短视野规划公式背后的直觉。

TD-MPC2 进一步强化了可扩展性和鲁棒性 [13]。当研究目标偏向机器人控制、连续控制或实体系统时，TD-MPC 这一支很值得关注。

TD-MPC 的价值在于实用。它不像生成式视频模型那样追求长期视觉真实感，也不像 MuZero 那样偏搜索博弈；它服务的是“接下来几步怎么控制更稳”。对于机械臂抓杯子、四足机器人走路、无人机避障这类任务，短视野里可靠通常比长视野里漂亮更重要。站在 2026 年看，TD-MPC 和 TD-MPC2 代表了世界模型走向连续控制工程化的一条关键路线。

6.3 Dreamer、MuZero 与 TD-MPC 的路线差异

路线	模型学什么	决策怎样发生	适合场景
----	-------	--------	------

PlaNet Dreamer	学 latent dynamics、奖励和终止信号，强调从观测到内部状态再到未来想象。	PlaNet 偏在线规划，Dreamer 偏在想象轨迹上训练 actor-critic。	像素控制、样本效率要求高的单智能体任务、需要从观测中学习状态的任务。
MuZero	学对搜索有用的抽象状态、动作后的隐状态变化、价值和策略。	在潜空间里做树搜索，用价值和策略指导分支选择。	棋类、Atari、离散动作空间、适合搜索的决策问题。
TD-MPC	学短视野内可靠的 latent dynamics 和 value 估计。	在有限 horizon 内优化一段候选动作序列，执行前几步后继重规划。	连续控制、机器人、机械系统、对实时稳定性要求高的任务。

可以把三条路线分别想成三种训练方式。Dreamer 像“先学会在脑内模拟，再把脑内练习变成直觉动作”；MuZero 像“在脑内展开几步棋，比较分支价值”；TD-MPC 像“每隔一小段时间重新算接下来几步怎样最稳”。它们都使用内部模型，但模型服务的对象不同：Dreamer 服务长期策略学习，MuZero 服务搜索式决策，TD-MPC 服务短视野连续控制。

7 Transformer、视频生成和可交互环境

7.1 序列建模的视角

语言模型的一个基本分解是：

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t}). \quad (20)$$

式中， p 表示概率分布或概率密度， $x_{1:T}$ 表示完整序列， x_t 表示第 t 个 token， $x_{<t}$ 表示 t 之前的 token， T 表示序列长度。其中， $\prod_{t=1}^T$ 表示从第 1 个位置到第 T 个位置的连乘， $p(x_t | x_{<t})$ 表示在历史 token 给定时预测当前 token 的条件分布。这个分解强调的是，模型按照时间顺序逐步预测后续内容。

一个三 token 例子：若模型给出 $p(x_1) = 0.5$ ， $p(x_2 | x_1) = 0.4$ ， $p(x_3 | x_1, x_2) = 0.25$ ，则整段序列概率为 $0.5 \times 0.4 \times 0.25 = 0.05$ 。连乘的含义就是把每一步条件预测串起来。

把这个思路放到世界模型里，动作条件版本常写成：

$$p(o_{1:T}, r_{1:T} | a_{1:T}) = \prod_{t=1}^T p(o_t, r_t | o_{<t}, a_{<t}). \quad (21)$$

式中， p 表示概率分布或概率密度， $o_{1:T}$ 表示观测序列， $r_{1:T}$ 表示奖励序列， $a_{1:T}$ 表示动作序列， $o_{<t}$ 表示过去观测， $a_{<t}$ 表示过去动作， $p(o_t, r_t | o_{<t}, a_{<t})$ 表示在历史观测和历史动作给定时预测当前观测与奖励的条件分布， $\prod_{t=1}^T$ 仍表示沿时间步连乘。这个写法说明，世界模型学的是受动作影响的序列分布。

一个两步例子：扫地机器人先前进再右转。模型预测第一步“到达沙发旁且奖励为 +1”的概率是 0.7，第二步“看到桌腿且奖励为 +1”的条件概率是 0.6，则这条两步观测和奖励片段的概率可以简化理解为 $0.7 \times 0.6 = 0.42$ 。动作条件改变后，这两个概率也会随之改变。

7.2 IRIS、GAIA-1、DriveDreamer、Genie

IRIS 把图像离散化后交给 Transformer 处理 [14]。它的价值在于，视觉世界模型可以借鉴 NLP 的 token 序列建模方式，但又要保留动作条件和控制目标。

IRIS 和 ViT 确实有相似之处：二者都会把图像变成 token，再用 Transformer 处理 token 之间的关系。差异在任务目标。ViT 常用于分类或表征学习，输入一张图后输出类别或 embedding；IRIS 服务世界模型，关心的是给定历史图像 token 和动作后，下一步图像 token、奖励和终止信号怎样变化。IRIS 打开了一个类比入口：图像可以先被切成离散 token，再交给 Transformer 预测未来。这个类比有用，但需要克制使用。语言 token 的生成主要沿文本上下文展开，世界模型里的 token 还受到动作、奖励和环境反馈影响。把视觉 token 当成“游戏里的词”，只能作为入门直觉，真正的难点仍然在交互闭环。

自动驾驶方向上，GAIA-1 和 DriveDreamer 都把生成式建模推进到了可控场景层面 [15, 16]。它们关心的重点包括场景一致性、未来轨迹、路况变化和动作条件生成。

Genie 的思路更进一步，尝试从互联网视频中学习可交互环境 [17]。这类工作把世界模型从传统 RL 语境带到更广的生成式模拟语境。如果一个模型能生成好看的视频，但动作与结果之间的关系不稳定，它离真正的可交互世界模型还有距离。

传统 RL 语境通常指智能体在一个明确环境里反复试错，例如 FrozenLake、CartPole、Atari 模拟器、MuJoCo 机器人控制。环境会接收动作，返回下一步观测、奖励和终止信号。生成式模拟语境则更接近“从大量视频和行为数据中学出一个可互动的环境生成器”。模型要能根据用户动作、规划轨迹或隐式动作，生成接下来会看到的画面和状态变化。前者的优势是评价清楚、因果闭环明确；后者的优势是数据来源更广，可以利用互联网上的大量视频和真实场景数据，服务开放世界模拟、机器人预训练和安全测试。

实现上通常有第四步。第一，把视频帧编码成离散 token 或连续 latent，降低直接处理像素的成本。第二，整理动作条件：若数据里有真实动作，就直接使用方向盘、速度、按键等控制信号；若普通视频没有动作标注，就学习 latent action，把相邻帧之间的可控变化压成离散或连续动作变量。第三，训练一个时序模型，根据过去 token、过去动作和当前动作预测未来 token。第四，把未来 token 解码成画面，或直接把 latent 交给策略、规划器、评估器使用。Genie 的特殊意义在于，它尝试在缺少显式动作标签的视频中推断这种可互动结构。

7.3 常见实验环境：Atari、Minecraft 与 ViZDoom

Atari 是世界模型和强化学习中最常见的图像游戏基准之一。它通常提供离散动作、低分辨率游戏画面、明确奖励和终止信号，适合比较样本效率、长期回报和模型规划能力。SimPLE、DreamerV2、MuZero、IRIS 和 DIAMOND 都大量使用 Atari。Atari 的优点是评测成熟、任务多、复现实验相对清楚；局限是画面和动作空间较规整，离开放世界交互还有距离。

Minecraft 代表更开放的三维沙盒环境。它包含第一人称视觉、连续视角控制、键鼠组合动作、物体采集、合成、背包、地形、建筑和长期任务。相比 Atari，Minecraft 更接近开放世界具

身智能：当前画面无法完整显示所有状态，背包和地图记忆会影响长期规划，动作也具有更复杂的组合结构。DreamerV3、MineDojo 和 MineWorld 都把 Minecraft 当作重要场景。

ViZDoom 是基于 DOOM 的第一人称射击研究环境 [4]。它提供第一人称视觉、离散或组合动作、快速移动、射击、导航和部分可观测性。World Models 早期就使用 Doom 任务展示从像素学习内部模型再训练控制器的路线；GameNGen 则把 DOOM 推到神经游戏引擎方向。ViZDoom 的价值在于动作反馈强、视觉变化快、场景结构比 Atari 更接近三维游戏，同时工程负担低于完整 Minecraft。

7.4 生成式交互环境工作对比

工作	技术侧重点	适用场景	局限或风险
IRIS	把视觉观测离散成 token，用 Transformer 做动作条件序列预测。	Atari 这类图像较规整、交互闭环清楚的任务。	离散 token 质量会限制预测上限，长时序一致性仍然困难。
GAIA-1	学习自动驾驶视频中的场景动态和动作条件生成。	驾驶场景模拟、数据增强、闭环测试前的离线预训练。	画面真实感需要和几何、交通规则、因果一致性共同验证。
Drive-Dreamer	强调可控驾驶视频生成，把道路结构、车辆运动和条件控制结合起来。	自动驾驶中的场景生成、稀有事件构造和感知模型测试。	控制条件与生成结果之间若缺少稳定对应，策略训练会受到误导。
Genie	从视频中学习可交互环境，尝试推断可操作的 latent action。	开放世界交互模拟、从无标注视频中构建可玩环境。	真实动作标签稀缺，长期交互稳定性和奖励定义仍然困难。

从发展脉络看，IRIS 把世界模型带向 Transformer token 序列；GAIA-1 和 DriveDreamer 把重点放到真实驾驶视频和可控生成；Genie 进一步尝试从普通视频中抽出可互动环境。这条线的共同意义在于扩大“世界模型”的数据来源和应用范围。早期 RL 世界模型依赖模拟器反复交互，生成式世界模型希望从海量被动视频中学习环境规律，再把这些规律变成可控、可查询、可用于训练的模拟器。

生成式世界模型在 2026 年仍处在快速变化中。它们的意义很大，因为机器人和自动驾驶都缺少足够多、足够安全、足够多样的真实交互数据。一个可控生成模型如果能可靠模拟“我向左打方向，车辆轨迹怎样变”，就能承担数据生成、压力测试和策略预训练的功能。它们的风险也很明显：画面真实不等于因果可靠，短视频合理不等于长时序稳定。

8 JEPA 路线

LeCun 的路线把世界模型、规划和表征学习放进同一个框架里 [20]。I-JEPA 处理图像表示学习 [21]，V-JEPA 2 则把视频理解、预测和规划进一步串起来 [22]。

JEPA 的关注点在于表征空间里的预测。它关心的是语义结构和可预测结构如何在 embedding 中保留下来。对世界模型研究来说，这条路线提供了一个重要提醒：

- 像素级高保真生成很重要。
- 语义级、控制级的可预测结构同样重要。

如果把 VAE 路线理解为“先压缩，再在潜空间里建模”，那么 JEPA 更像“在表征空间里直接组织可预测结构”。这两条路线在目标上有交叉，但关注点存在差异。JEPA 缓解了像素级高保真重建带来的负担。它通常预测 embedding，跳过逐像素预测，所以模型可以把容量放在对象、运动、遮挡、语义关系这类结构上。代价也很直接：embedding 预测本身不会自然给出一段高清可播放视频。若任务需要可视化仿真、数据生成或闭环视频环境，仍然需要额外的生成模块或解码模块。若任务更关心理解、规划和表示迁移，JEPA 风格的表征预测就很有吸引力。

可以用“记路线”和“画照片”来类比。VAE 或视频生成路线更像学会把街景照片画出来；JEPA 更像学会记住“路口、车道、行人、车辆运动趋势”这些结构关系。画照片能检查细节保留情况，记结构更贴近规划。世界模型研究到 2026 年的一条重要趋势，就是在这两类目标之间寻找合适组合：既保留足以控制的结构，又在需要时生成足够真实的未来观测。

9 DINO/DINOv2 与非生成式视觉表征

前面的主线主要讨论“如何预测未来”。但在视觉世界模型里，还有一个更靠前的问题：当前画面应当怎样被编码。DINO 和 DINOv2 属于视觉自监督表征学习路线 [18, 19]。它们主要学习视觉表征，缺少动作条件动力学、奖励预测和终止信号预测，因此更适合被看作视觉表征模块。Dreamer、PlaNet 或 MuZero 意义上的完整世界模型还需要额外的动态建模与决策接口。但它们解决了世界模型中的一个基础问题：如何从高维图像中得到稳定、语义化、可迁移的视觉表示。

先看 DINO 的训练目标。给定一张图像 x ，数据增强会产生多个视图，包括两个 global crop 和若干 local crop。学生网络 f_{θ_s} 接收所有视图，教师网络 f_{θ_t} 通常只接收 global crop。二者后面都接一个 projection head，输出一组 prototype logits。对某个学生视图 v 和教师视图 u ，可以写成：

$$p_s^{(v)} = \text{softmax}\left(\frac{g_s(v)}{\tau_s}\right), \quad p_t^{(u)} = \text{softmax}\left(\frac{g_t(u) - c}{\tau_t}\right). \quad (22)$$

式中， $g_s(v)$ 和 $g_t(u)$ 分别是学生和教师 head 的 logits； τ_s 与 τ_t 是温度系数； c 是 teacher 输出的中心项。温度越小，softmax 越尖锐；中心项负责把 teacher 的输出整体平移，避免某几个维度长期占优。这里的 $p_s^{(v)}$ 和 $p_t^{(u)}$ 表示投影头输出到一组 prototype 后得到的分布，含义接近“这张增强视图在当前表征空间里靠近哪些原型方向”。可以把每个 prototype 理解成当前表征空间里的一个软聚类方向：某张图像越接近某个方向，对应维度的概率就越大。 v 和 u 表示同一张图像经过不同增强得到的两个视图，可能是 global crop，也可能是 local crop；DINO 通常让 teacher 看 global crop，让 student 看全部视图。因此，式 (22) 的核心目标是把不同视图都变成可比较的 prototype 分布。它没有使用人工类别标签，训练信号来自同一图像不同增强视图之间的一致性。

τ_s 和 τ_t 的作用是控制分布的熵。如果温度太高, softmax 会过于平坦, 所有 prototype 看起来差不多, 目标缺少辨识度; 如果温度太低, 分布会过尖, 训练早期容易被噪声牵引。DINO 通常让 teacher 的温度更低, 使 teacher 目标更尖锐, 让 student 有明确的对齐方向。中心项 c 则是防 collapse 的稳定器之一: 若某些 prototype 维度长期偏大, teacher 会反复偏向这些维度, 所有图像可能被吸到少数输出上; 减去滑动平均中心后, 输出空间会更均衡。所以式 (22) 实际上是在构造一个稳定、可对齐、不过度塌缩的自监督目标分布。输入是增强图像视图, 输出是 prototype 概率分布; 这个输出随后会被交叉熵损失拿来对齐 student 和 teacher。

DINO 的核心损失是让学生在在一个视图上的预测去匹配教师在另一个视图上的预测:

$$\mathcal{L}_{\text{DINO}} = \sum_{u \in \mathcal{V}_g} \sum_{\substack{v \in \mathcal{V} \\ v \neq u}} H\left(\text{sg}\left[p_t^{(u)}\right], p_s^{(v)}\right) = - \sum_{u,v} \sum_k p_{t,k}^{(u)} \log p_{s,k}^{(v)}. \quad (23)$$

这里 $\mathcal{V}_g$ 表示 global crop 集合, $\mathcal{V}$ 表示全部视图集合, $H(\cdot, \cdot)$ 是交叉熵, $\text{sg}[\cdot]$ 表示 stop-gradient。直观地说, 教师给出“这张图在当前表示空间里应该落到哪里”的软目标; 学生需要在不同裁剪、不同增强下给出一致预测。这是一种无人工标签的自监督对齐: 同一图像的不同视图需要在表征空间中靠近。更具体地说, 外层求和先枚举 teacher 看到的 global view u , 内层再枚举 student 看到的其他视图 v 。条件 $v \neq u$ 的意思是避免 student 只复述同一个输入视图, 训练信号来自跨视图预测。交叉熵 $H(\text{sg}[p_t^{(u)}], p_s^{(v)})$ 会在 $p_t^{(u)}$ 权重大的 prototype 上强烈惩罚 student 的低概率输出。等价写法 $-\sum_{u,v} \sum_k p_{t,k}^{(u)} \log p_{s,k}^{(v)}$ 展开了这个过程: k 是 prototype 维度, teacher 认为重要的维度, student 也必须给出较高概率。

从输入输出看, 式 (23) 接收的是多视图增强后的 teacher/student 概率分布, 输出是一个标量损失。损失下降意味着: 同一图像即使经过裁剪、颜色扰动或局部遮挡, 也会被编码到相近的 prototype 结构中。这正是 DINO 在分割、检索和迁移任务中表现好的关键原因。

stop-gradient 是这条损失能工作的关键。 $\text{sg}[p_t^{(u)}]$ 表示 teacher 分布在当前反向传播中被当成常量, 梯度只推动 student 去靠近 teacher。teacher 本身不被式 (23) 直接更新, 后面通过 momentum update 从 student 平滑得到。如果去掉这个不对称结构, 两个网络可能一起追逐一个不断移动的目标, 甚至一起滑向所有样本输出相同分布的 collapse。因此, 式 (23) 可以理解为“固定一个较稳定的 teacher 目标, 让 student 在各种增强视图下追上它”。

放到 game world model 的语境里, 式 (23) 的价值在于逼视觉编码器学习跨视角、跨裁剪仍然稳定的对象和场景表征。例如同一个房间被裁掉一部分、角色被局部遮挡、道具出现在不同位置时, 模型仍然应当把它们映射到相近的语义区域。这类表征本身尚未构成动作条件动力学, 但它能给后续 dynamics model 提供更干净的输入, 减少模型把光照、纹理、裁剪差异误当成状态变化的风险。

教师网络由学生网络的指数滑动平均得到, 不通过反向传播直接更新:

$$\theta_t \leftarrow m\theta_t + (1-m)\theta_s. \quad (24)$$

其中 m 通常随训练逐渐接近 1。这个 momentum teacher 是 DINO 的关键稳定器: 学生更新较快, 教师更新较慢, 教师因此提供一个更平滑的目标。若教师也跟着梯度快速变化, 训练目标会剧烈摆动; 若没有 teacher-student 不对称, 模型更容易陷入所有图像输出同一分布的 collapse。

中心项 c 也用滑动平均更新:

$$c \leftarrow \mu c + (1-\mu) \frac{1}{B} \sum_{b=1}^B g_t(x_b). \quad (25)$$

式中 B 是 batch size, μ 是中心项的动量系数。centering 会抑制某些 prototype 维度被长期偏置占满; sharpening 则通过较小的 teacher temperature 让目标分布更有区分度。二者配合起来, 既避免输出过于平均, 也避免全部样本塌到同一个 prototype。

DINO trick	做法	作用
多裁剪训练	同一图像生成 global crop 和 local crop; teacher 看 global, student 看全部视图。	让局部 patch 也对齐全局语义, 逼迫模型学习对象级结构, 减少只记纹理的风险。
momentum teacher	teacher 参数是 student 参数的指数滑动平均。	提供稳定目标, 缓解训练目标自我追逐。
stop-gradient	teacher 分支不回传梯度。	保持 teacher 作为目标网络, 避免两个分支同时塌缩。
centering	对 teacher logits 减去滑动平均中心 c 。	防止少数输出维度长期占优, 保持 prototype 使用更均衡。
sharpening	teacher 使用较低温度 τ_t 。	让教师目标更有辨识度, 避免输出过于平。
小 patch ViT	使用较小 patch 时, self-attention 更容易形成对象区域。	对分割、定位和局部匹配更友好。

可以把 DINOv2 放在“视觉 backbone”这一层理解。如果一个视觉编码器只保留纹理和颜色, 后续 latent dynamics 很难学到物体、空间关系和可交互结构。如果编码器已经能够稳定表示物体边界、场景布局、语义区域和几何线索, 世界模型就可以把更多容量用于学习时间变化、动作影响和规划相关因素。因此, DINOv2 的意义在于给世界模型、VLA 和机器人策略提供更好的输入表示; 完整世界模型仍然需要 dynamics 与动作建模。

DINOv2 的贡献可以理解为把 DINO 这类自监督视觉表征训练扩展到 foundation model 规模。它的提升来自一组训练组件的组合: 图像级 DINO loss、patch 级 iBOT masked prediction、特征分布正则、数据自动清洗和蒸馏。其中, iBOT 部分可以粗略理解为“遮住一部分 patch, 让学生预测 teacher 在这些位置上的表征目标”:

$$\mathcal{L}_{\text{iBOT}} = \sum_{i \in \mathcal{M}} H(\text{sg}[p_{t,i}], p_{s,i}), \quad (26)$$

其中 $\mathcal{M}$ 是被 mask 的 patch 位置集合。这使模型不只学整图 class token, 也学 patch-level 的局部结构。对机器人和游戏世界模型来说, 局部结构很重要: 角色、门、道具、敌人、可通行区域和障碍物都不一定能由一个整图向量充分表达。

DINOv2 还加入了让特征在 batch 内分布更均匀的 KoLeo regularizer。简化写法可以表示为:

$$\mathcal{L}_{\text{KoLeo}} = -\frac{1}{B} \sum_{i=1}^B \log \rho_i, \quad \rho_i = \min_{j \neq i} \|z_i - z_j\|_2. \quad (27)$$

这里 z_i 是归一化后的图像特征, ρ_i 是它到 batch 中最近邻特征的距离。这个正则鼓励特征不要挤在一起, 避免表示空间过度拥挤。在检索、匹配、分割和跨场景迁移任务中, 更均匀的特征空

间通常更容易使用。

DINOv2 组件	作用	对世界模型的启发
DINO image-level loss	对齐同一图像不同视图的全局语义。	提供稳定的 scene/object-level 表示。
iBOT patch-level loss	让 masked patch 去匹配 teacher 的局部表征。	帮助保留局部物体、区域和可交互结构。
KoLeo regularizer	拉开 batch 内特征，鼓励分布更均匀。	减少不同状态挤到同一 latent 区域的风险。
数据清洗与去重	从大规模未标注图像中筛出多样、质量较高的数据。	提醒 game world model 不能只堆视频数量，还要控制数据分布和重复度。
蒸馏与高分辨率适配	先训练大模型，再转移到较小模型或更高分辨率输入。	适合把强视觉 backbone 接入资源受限的策略或世界模型。

OpenVLA 这类机器人基础模型已经体现出这种组合方式 [24]。它借助强视觉编码器和语言模型，再用机器人演示数据学习动作输出，避免从零学习所有视觉特征。这说明未来的具身系统更可能由视觉表征、语言理解、动作策略和世界预测共同组成。

路线	主要回答的问题	和世界模型的关系
DINO/DINOv2	当前图像怎样编码成可迁移表示。	提供强视觉 backbone，帮助后续动力学和策略少受低层纹理干扰。
I-JEPA/V-JEPA	在 embedding 空间里预测被遮挡或未来结构。	直接靠近“表征空间中的预测”，与世界模型目标更接近。
Dreamer/PlaNet	动作条件下 latent state 怎样演化。	完整建模交互闭环，服务规划和策略学习。
Genie/GameNGen	如何生成可交互环境或神经游戏引擎。	把世界模型推向可玩、可控、可视化的模拟器。

对 game world model 研究来说，DINO/DINOv2 值得作为背景阅读。游戏画面中有大量纹理、光照、UI 和快速运动，世界模型若把容量都花在复原这些低层细节上，可能会忽略真正影响交互的对象、规则和空间关系。强视觉表征不能自动解决动作建模问题，但它能让后续模型从更干净的状态描述起步。在 game world model 中，这一点尤其直接。如果编码器能稳定识别角色、门、道路、敌人、道具、血条和背包 UI，后续世界模型就更容易学习“按下攻击键会改变哪个对象”“进入门后场景如何切换”“拾取物品后背包状态如何变化”。反过来，如果编码器只盯着像素纹理，模型可能生成看起来相似的下一帧，却不理解规则状态已经改变。

因此，DINO/DINOv2 对世界模型的意义可以概括为三句话。第一，它们提供了无标签图像预训练的强视觉表征，降低了世界模型从零学习视觉结构的压力。第二，它们让“表征是否保

留对象和空间关系”成为一个可讨论的问题，使评价视角超出重建误差。第三，它们提醒 game world model 的研究者：可交互世界不只需要生成未来，还需要把当前世界编码成适合预测、控制和评测的状态。

10 具身智能研究版图：世界模型、VLA 与 LAM

近两年，具身智能讨论中常出现三类关键词：世界模型、VLA 和 LAM。这个划分有帮助，但不能当成严格 taxonomy。更稳妥的理解方式，是把具身智能看成一个闭环系统：感知形成状态，推理解目标，策略生成动作，世界模型预测后果，数据和评测保证能力可迁移、可复现、可落地。

世界模型关注环境动力学。它试图回答：如果智能体在当前状态下采取某个动作，未来观测、奖励、可行性或场景结构会怎样变化。Dreamer、TD-MPC、MuZero、Genie、GAIA-1 和 DriveDreamer 都可以从不同角度放进这条线。世界模型的价值在于让智能体能够在内部进行预测、规划和训练，减少对真实世界试错的依赖。

VLA，即 Vision-Language-Action model，关注从视觉和语言到动作的端到端映射。RT-2 把互联网规模视觉语言预训练和机器人动作输出结合起来，把动作也表示成模型可生成的 token [23]。OpenVLA 则把 VLA 做成开源路线，强调多机器人演示数据、视觉语言模型和机器人动作之间的结合 [24]。 π_0 进一步用 flow matching 生成连续动作，代表 VLA 从离散 token 输出走向高频连续控制的一条路线 [25]。VLA 更像具身智能中的通用策略模型，重点是把“看见什么、听到什么”转化为“接下来做什么”。

LAM 需要区分语境。如果指 Large Action Model，它通常强调动作序列生成、工具调用、机器人执行和反馈闭环，是比 VLA 更宽泛的行动模型概念。如果指 Latent Action Model，它则常出现在 Genie 这类系统中，用于从无动作标注的视频中推断潜在动作，帮助模型获得可控制的交互接口 [17]。这两个概念名字相近，但研究对象不同。

方向	核心问题	典型交集
感知与表征学习	当前世界怎样被编码。	DINOv2、JEPa、VLA 视觉编码器、世界模型 encoder。
任务理解与规划	用户目标怎样分解成步骤。	LLM/VLM planner、task and motion planning、世界模型 lookahead。
动作策略与技能学习	具体输出什么动作。	VLA、Diffusion Policy、Large Action Model、模仿学习和强化学习。
世界模型与预测仿真	动作之后世界怎样变化。	Dreamer、TD-MPC、Genie、GameNGen、自动驾驶世界模型。
数据、迁移与安全评测	能力怎样规模化和验证。	Open X-Embodiment、benchmark、真机评测、仿真到现实迁移。

从系统角度看，三者最终更可能融合，替代关系反而较弱。一个具身智能体可以用 DINOv2

或 JEPA 风格模型形成视觉表示，用 VLA 理解指令并生成初步动作意图，用动作模型把意图转成可执行轨迹，再用世界模型预测风险和后果。在这个组合里，世界模型不一定站在最前台，但它承担“先想再做”的功能。

11 Benchmark: 具身智能进展如何被衡量

Benchmark 是正文级问题。在具身智能里，一个模型是否真正具备泛化能力，不能只靠几个演示视频判断。它需要在明确任务、数据划分、评价指标和可复现实验协议下接受测试。因此，benchmark 不只是评测工具，也在定义研究问题本身。

早期世界模型主要依赖 Atari、DeepMind Control Suite、DMLab、ProcGen、BSuite 等强化学习环境。这些环境可重复、成本低，适合比较样本效率、长期回报和规划能力。Dreamer、MuZero、TD-MPC 等工作正是在这些 benchmark 上形成了清晰的性能比较。但这类环境和真实机器人之间仍有距离，特别是在接触动力学、传感器噪声和开放物体分布方面。

具身导航和家庭任务 benchmark 把语言、视觉和行动结合起来。ALFRED 要求智能体根据自然语言指令在家庭环境中完成长时程任务 [27]。BEHAVIOR 则把 everyday household activities 组织成更复杂的家庭活动基准 [28]。这类 benchmark 的意义在于：语言必须落实到环境状态变化和动作序列上。

机器人操作 benchmark 更接近真实具身系统。RLBench 提供大量机器人操作任务和自动生成演示 [29]。ManiSkill2 强调多任务操作、物体泛化和统一评测接口 [30]。CALVIN 用于语言条件长时程操作 [31]。LIBERO 则把重点放在 lifelong robot learning 和 knowledge transfer 上 [32]。VLA、Diffusion Policy、LAM 和机器人基础模型都需要在这类 benchmark 上证明动作生成能力。

随着通用机器人模型发展，benchmark 还需要关注跨任务、跨场景和跨机器人本体的迁移。Open X-Embodiment 把不同实验室、不同机器人和不同任务的数据合并起来，并提出 RT-X 模型作为跨本体学习的尝试 [26]。这类工作提示：通用具身智能的难点不仅在任务数量，还在传感器配置、动作空间和机器人形态之间如何对齐。

对生成式世界模型来说，benchmark 仍不够成熟。视频质量指标只能说明生成结果是否像真实视频，却不能充分说明动作条件是否可靠、长期预测是否稳定、生成环境是否能用于训练智能体。因此，未来世界模型 benchmark 需要同时评价视觉真实性、动作可控性、长期一致性和下游控制收益。

Benchmark 类型	常见指标	容易忽略的问题
RL/世界模型环境	return、sample efficiency、planning performance。	分数高不等于模型理解因果结构，游戏或仿真可能存在 shortcut。
具身导航/家庭任务	success rate、SPL、任务完成率、语言泛化。	语言模板和场景偏置可能让模型绕过真正理解。
机器人操作	成功率、长时程完成率、跨物体/跨场景泛化。	rollout 数量少时统计波动大，固定桌面设置容易过拟合。

VLA/机器人基础模型	多任务成功率、跨机器人迁移、真机成功率。	不同动作空间和机器人形态使比较很难完全公平。
生成式世界模型	FVD、动作响应、规则一致性、长期记忆。	画面真实不等于可玩，也不等于能训练智能体。

阅读 benchmark 时建议把评测协议当成正文来读。至少问五个问题：训练和测试是否真的分开；任务是否有 shortcut；是否报告多随机种子或置信区间；是否测试新物体、新场景或新机器人；指标是否和最终目标一致。如果一篇论文只展示漂亮视频或少量 cherry-picked demo，就不能把它当成已经解决通用具身智能问题。

12 Game World Model: 从游戏智能体到神经游戏引擎

Game world model 是世界模型研究中正在快速发展的分支。它聚焦游戏或类游戏交互环境，学习视觉状态、动作后果、规则结构、奖励机制和长期交互动态。与一般视频生成不同，game world model 必须关心动作可控性：玩家或智能体采取一个动作后，环境需要按照稳定、可预测、可交互的方式变化。

游戏是研究世界模型的理想试验场。一方面，游戏环境可以快速重置、低成本采样，并且天然提供观测、动作、奖励和事件反馈。另一方面，复杂游戏又包含长期探索、空间导航、资源收集、工具使用和组合技能等问题，足以逼近具身智能中的许多核心难点。因此，游戏世界模型既服务于强化学习智能体训练，也逐渐走向神经游戏引擎和交互式内容生成。

早期世界模型更多把游戏作为智能体训练环境。World Models 在 CarRacing 和 Doom 中展示了先学习内部模型、再训练控制器的路线 [3]。Dreamer、MuZero、IRIS 和 DIAMOND 等工作进一步说明，智能体可以在潜空间、抽象状态或生成模型中进行想象、搜索或策略学习 [8, 11, 14, 33]。这里的核心指标是游戏分数、样本效率和从模型到真实游戏环境的迁移效果。

近年的另一条路线是神经游戏引擎。GameNGen 用 diffusion 模型模拟 DOOM，根据历史画面和玩家动作实时生成下一帧，使游戏可以在神经模型中交互运行 [34]。Genie 则从无标签互联网视频中学习生成式交互环境，通过 latent action model 获得可控制的潜在动作接口 [17]。传统游戏引擎显式编写碰撞、渲染和规则逻辑；这类模型从数据中学习这些机制的统计近似。

Minecraft 和开放世界游戏为 game world model 提供了更复杂的研究场景。MineDojo 将 Minecraft 与互联网规模的视频、教程和 wiki 知识结合，用于研究开放式具身智能体 [36]。Craftax 则提供更轻量的生存游戏环境，用于探索、长期规划和开放式强化学习 [37]。这类环境的重点从“玩 Atari 拿高分”转向开放世界建模，尤其是可交互结构、物体持久性、工具链、背包状态和长期目标。

MineWorld 则更直接地把 Minecraft 建成可交互世界模型 [35]。它把游戏画面和玩家动作都转换成离散 token：视觉 tokenizer 把每帧图像压成视觉 token，action tokenizer 把键盘、鼠标、相机旋转等控制信号变成动作 token，再把二者交错拼接后交给自回归 Transformer。这种设计使模型同时学习两类关系：游戏状态之间怎样演化，以及动作怎样改变下一步视觉状态。与只生成视频的模型相比，MineWorld 的研究重点更接近 game world model 的核心：生成结果必须跟随输入动作，并且推理速度要接近可交互。

从算法结构看, game world model 至少需要拆成四个模块。第一是视觉表示模块, 负责把画面压成可预测的 token 或 latent。第二是动作表示模块, 负责把键盘、鼠标、手柄或高层指令整理成模型可用的条件。第三是动态模型, 负责学习 $p(x_{t+1} | x_{\leq t}, a_{\leq t})$ 或对应的 latent/token 版本。第四是解码与交互模块, 负责把预测状态还原成画面, 并在低延迟条件下接收下一步动作。这四个模块中, 任何一环失真都会破坏可玩性: 视觉 token 太粗会丢物体细节, 动作 token 太粗会丢控制差异, 动态模型太弱会造成长期漂移, 解码太慢会失去交互价值。

方向	代表问题	评价重点
游戏智能体世界模型	在模型里训练或规划, 减少真实游戏交互。	游戏分数、样本效率、真实环境迁移。
神经游戏引擎	用神经网络替代部分渲染和规则更新。	实时性、动作响应、画面稳定性、长期一致性。
无动作标签视频学习	从游戏视频中恢复可控 latent action。	潜在动作是否稳定、是否可玩、是否能组合。
开放世界游戏模型	建模 Minecraft/Crafter 类长期交互世界。	地图记忆、物体持久性、工具使用、任务进度。
游戏内容生成	生成关卡、地图、玩法原型或可探索空间。	可玩性、多样性、规则一致性和创作可控性。

评价 game world model 不能只看画面是否逼真。一个真正可用的游戏世界模型至少需要同时满足六类要求: 视觉质量、动作可控性、规则一致性、长期状态记忆、实时交互速度, 以及下游智能体训练收益。若模型生成的视频很清晰, 但玩家按下跳跃后角色不稳定地漂移, 或者刚放下的物体几秒后消失, 它仍然缺乏游戏世界模型所需的可靠性。

MineWorld 的评估给出了一个有参考价值的拆法。视频质量可以用 FVD、LPIPS、SSIM、PSNR 衡量; 动作可控性则可以用逆动力学模型 IDM 从生成帧中反推动作, 再和原始输入动作比较。离散动作部分可计算 precision、recall 和 F1, 相机控制部分可计算预测相机 bin 与真实相机 bin 的 L1 距离。这个思路很重要: 视频像不像真实游戏, 和动作有没有被执行, 是两个不同问题。

因此, game world model 的实验表不宜只报告一组视频指标。更合理的结构是把指标分成几列: 视觉质量、动作跟随、长期一致性、实时速度和下游收益。还需要保留 no-action、correct-action、shuffled-action 等对照。如果输入动作被打乱后模型的 IDM F1 几乎不下降, 说明模型可能主要依赖历史帧惯性, 并没有真正使用动作条件。

当前 game world model 仍有几个明显难点。第一是长期一致性, 模型需要记住地图、物体、背包和任务进度, 而不仅是生成下一帧。第二是动作因果性, 玩家动作应该稳定改变世界状态, 避免只造成视觉上合理但规则上混乱的变化。第三是隐藏状态建模, 许多游戏状态不会完整显示在当前画面中, 例如敌人位置、库存、任务标志和冷却时间。第四是实时性, 可交互世界模型需要低延迟生成, 而高质量视频模型通常计算昂贵。第五是评价体系, FVD 等视频指标不能充分衡量可玩性、规则一致性和下游训练价值。第六是数据和版权, 大量游戏视频可用于训练, 但游戏资产、玩家数据和商业版权会带来额外约束。

从研究选题角度看, game world model 可以视为数字具身智能的训练场。它比真实机器人便宜、安全、可重复, 又比静态视觉任务更接近真实交互闭环。真正值得抓住的是一个可控制、可记忆、可规划、可评测的交互世界, 而不只是好看的游戏画面。

13 一张时间线

年份	代表工作	脉络意义
1991	Dyna [1]	把学习、规划和反应放到同一个循环里。
2013/2014	VAE [2]	给 latent space 加上概率结构, 后来成为视觉世界模型的重要工具。
2018	<i>World Models</i> [3]	用 VAE、RNN 和控制器展示从像素到内部世界的路线。
2019	SimPLe [5]	将视频预测式模型用于 Atari。
2019	MBPO [6]	突出短 rollout 的样本效率收益。
2019	PlaNet [7]	在 latent space 里学习动力学并做规划。
2019	Dreamer [8]	用 imagined trajectory 训练策略。
2020	MuZero [11]	学习对规划有用的抽象模型。
2020	DreamerV2 [9]	引入离散 latent 表示。
2022	TD-MPC [12]	把 TD 学习与 MPC 结合。
2022	IRIS [14]	用 Transformer 建世界模型。
2022	MineDojo [36]	把 Minecraft、互联网知识和开放式具身智能连接起来。
2023	DreamerV3 [10]	强调跨领域通用性。
2023	GAIA-1 / Drive-Dreamer [15, 16]	自动驾驶中的生成式世界模型。
2023	DINOv2 / Open X-Embodiment [19, 26]	强视觉表征和跨机器人数据开始成为具身智能基础设施。
2023	RT-2 [23]	VLA 路线把视觉语言知识迁移到机器人动作输出。
2024	Genie [17]	交互式环境生成开始成为独立方向。
2024	DIAMOND / GameN-Gen [33, 34]	游戏世界模型走向 diffusion world model 和神经游戏引擎。
2024	OpenVLA / π_0 [24, 25]	开源 VLA 与连续动作生成推动机器人基础模型发展。
2024	世界模型综述 [38]	总结“理解世界”与“预测未来”的不同侧重。

2025	MineWorld [35]	将 Minecraft 建成开源实时交互式世界模型，并突出动作可控性评估。
2025	V-JEPA 2 [22]	自监督视频表征进一步连接理解、预测和规划。
2026	机器人与具身智能综述 [39, 40]	世界模型扩展到机器人学习和具身智能。

14 阅读路线

建立完整直觉时，不宜平均阅读所有论文，更适合按研究问题分层推进。下面这条路线保留世界模型主线，同时补上视觉表征、具身智能、benchmark 和 game world model 四个扩展方向。

14.1 第一层：世界模型主线必读

1. 先读 *World Models* [3]，把 VAE + RNN + controller 的图看懂。
2. 再读 PlaNet [7]，理解 latent dynamics 和规划。
3. 然后读 Dreamer [8]，把 imagined trajectory 的训练方式吃透。
4. 接着读 MuZero [11] 和 TD-MPC [12]，理解规划与控制的两种实用路线。
5. 最后读 IRIS、Genie、GAIA-1 和 DriveDreamer [14, 17, 15, 16]，理解世界模型如何走向 token 序列、可交互环境和真实场景生成。

每读一篇论文，可以带着下面这些问题：

- 模型学到的状态是什么。
- 它预测什么。
- 动作如何进入模型。
- 模型误差怎样被控制。
- 最终服务的是规划、控制、生成，还是表征学习。
- 评估指标是否真正反映了决策收益。

14.2 第二层：视觉表征和具身智能补强

视觉表征补强建议阅读 DINOv2、I-JEPA 和 V-JEPA 2 [19, 21, 22]。这三篇属于表征学习路线，能解释现代世界模型为什么越来越重视“表征空间中的预测”。DINOv2 重点看视觉特征如何自监督形成；I-JEPA 重点看图像 embedding 中的预测结构；V-JEPA 2 重点看视频表征如何进一步服务理解、预测和规划。

具身智能方向建议读 RT-2、OpenVLA、 π_0 和 Open X-Embodiment [23, 24, 25, 26]。RT-2 适合理解 VLA 的基本范式；OpenVLA 适合理解开源机器人基础模型的工程形态； π_0 适合理解连续动作生成；Open X-Embodiment 适合理解大规模机器人数据和跨本体迁移为什么困难。

14.3 第三层：benchmark 和评测

Benchmark 论文建议按任务类型读。具身导航和家庭任务可以读 ALFRED 与 BEHAVIOR [27, 28]。机器人操作可以读 RLBench、ManiSkill2、CALVIN 和 LIBERO [29, 30, 31, 32]。阅读这些论文时，重点放在观测空间、动作空间、训练测试划分、成功率定义、随机种子和泛化设置。

对世界模型研究来说，benchmark 的核心问题是：模型预测是否真的能带来控制收益。如果一个模型 FVD 很低，却不能让 agent 拿到更高 return，或者不能稳定响应动作，它仍未达到可靠交互世界模型的标准。

14.4 第四层：game world model 重点阅读

game world model 方向建议把阅读重点放在下面几组：

- **基础游戏智能体世界模型：** *World Models*、Dreamer、MuZero、IRIS [3, 8, 11, 14]。目标是理解从游戏观测到内部状态、再到规划或策略学习的闭环。
- **生成式游戏世界模型：** DIAMOND、GameNGen、Genie、MineWorld [33, 34, 17, 35]。目标是理解 diffusion/video/autoregressive model 如何变成可交互模拟器。
- **开放世界游戏环境：** MineDojo、Craftax、Minecraft-like 任务 [36, 37]。目标是理解长期探索、工具使用、资源收集和开放式任务。
- **评价指标：** 除了 return 和 FVD，还要看动作可控性、规则一致性、长期记忆、实时性和下游智能体收益。

这条阅读路线的目标是建立可研究的问题图谱，无需把所有论文平均读完。对 game world model 来说，真正值得长期跟踪的问题是：如何从游戏数据中学到可控制、可记忆、可规划、可评测的交互世界。

结语

世界模型的发展可以看成三层推进。

第一层，是 Dyna 到深度 model-based RL，重点在于环境规律如何被学习和复用。第二层，是 latent world model 和 Dreamer 主线，重点在于如何把高维观测压到可控的内部空间，再在内部空间里想象未来。第三层，是 MuZero、TD-MPC、视频生成、自动驾驶、JEPA、VLA 和 game world model 路线，重点在于世界模型如何服务于规划、控制、生成和具身智能。

以 2026 的眼光重新分配篇幅，最值得抓牢的部分有六块。第一块是 VAE、RSSM、latent state 这类底层结构，因为它们决定了世界模型为何能在高维观测下工作。第二块是 Dreamer 系列，因为它把内部想象和策略学习真正做成了通用框架。第三块是 MuZero 和 TD-MPC，因为它们分别代表“为搜索服务的抽象建模”和“为连续控制服务的短视野规划”。第四块是 DINO、JEPA、VLA 和 LAM 所代表的具身智能相邻路线，因为它们说明世界模型和视觉表征、语言理解、动作模型紧密相连。第五块是 benchmark，因为评测协议决定了到底在比较什么能力。第六

块是 game world model，因为它既有清晰的交互闭环，又连接生成式视频、神经游戏引擎、开放世界智能体和数字具身环境。

把时间集中到这些核心部分上，收益通常高于平均阅读每一篇论文。对应地，时间线里的辅助工作适合保留作为坐标，不适合占掉太多正文篇幅。

当前最值得先抓牢的主线是：观测表征 -> 动作条件动力学 -> 未来预测 -> 规划与控制 -> 交互式游戏世界评测。沿着这条线阅读论文，更容易把不同工作放进同一张图里。

参考文献

- [1] Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 1991.
- [2] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. arXiv:1312.6114, 2013; ICLR 2014. <https://arxiv.org/abs/1312.6114>
- [3] David Ha and Jurgen Schmidhuber. World Models. arXiv:1803.10122, 2018. <https://arxiv.org/abs/1803.10122>
- [4] Michal Kempka et al. ViZDoom: A Doom-based AI Research Platform for Visual Reinforcement Learning. arXiv:1605.02097, 2016. <https://arxiv.org/abs/1605.02097>
- [5] Lukasz Kaiser et al. Model-Based Reinforcement Learning for Atari. arXiv:1903.00374, 2019. <https://arxiv.org/abs/1903.00374>
- [6] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to Trust Your Model: Model-Based Policy Optimization. arXiv:1906.08253, 2019. <https://arxiv.org/abs/1906.08253>
- [7] Danijar Hafner et al. Learning Latent Dynamics for Planning from Pixels. arXiv:1811.04551, 2019. <https://arxiv.org/abs/1811.04551>
- [8] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. arXiv:1912.01603, 2019. <https://arxiv.org/abs/1912.01603>
- [9] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering Atari with Discrete World Models. arXiv:2010.02193, 2020. <https://arxiv.org/abs/2010.02193>
- [10] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering Diverse Domains through World Models. arXiv:2301.04104, 2023. <https://arxiv.org/abs/2301.04104>
- [11] Julian Schrittwieser et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588:604–609, 2020. <https://www.nature.com/articles/s41586-020-03051-4>

- [12] Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal Difference Learning for Model Predictive Control. arXiv:2203.04955, 2022. <https://arxiv.org/abs/2203.04955>
- [13] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. arXiv:2310.16828, 2023. <https://arxiv.org/abs/2310.16828>
- [14] Vincent Micheli, Eloi Alonso, and Francois Fleuret. Transformers are Sample-Efficient World Models. arXiv:2209.00588, 2022. <https://arxiv.org/abs/2209.00588>
- [15] Anthony Hu et al. GAIA-1: A Generative World Model for Autonomous Driving. arXiv:2309.17080, 2023. <https://arxiv.org/abs/2309.17080>
- [16] Xiaofeng Wang et al. DriveDreamer: Towards Real-world-driven World Models for Autonomous Driving. arXiv:2309.09777, 2023. <https://arxiv.org/abs/2309.09777>
- [17] Jake Bruce et al. Genie: Generative Interactive Environments. arXiv:2402.15391, 2024. <https://arxiv.org/abs/2402.15391>
- [18] Mathilde Caron et al. Emerging Properties in Self-Supervised Vision Transformers. arXiv:2104.14294, 2021. <https://arxiv.org/abs/2104.14294>
- [19] Maxime Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193, 2023. <https://arxiv.org/abs/2304.07193>
- [20] Yann LeCun. A Path Towards Autonomous Machine Intelligence. OpenReview, 2022. <https://openreview.net/forum?id=BZ5a1r-kVsf>
- [21] Mahmoud Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. arXiv:2301.08243, 2023. <https://arxiv.org/abs/2301.08243>
- [22] Mido Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025. <https://arxiv.org/abs/2506.09985>
- [23] Anthony Brohan et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818, 2023. <https://arxiv.org/abs/2307.15818>
- [24] Moo Jin Kim et al. OpenVLA: An Open-Source Vision-Language-Action Model. arXiv:2406.09246, 2024. <https://arxiv.org/abs/2406.09246>
- [25] Kevin Black et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024. <https://arxiv.org/abs/2410.24164>
- [26] Open X-Embodiment Collaboration et al. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv:2310.08864, 2023. <https://arxiv.org/abs/2310.08864>
- [27] Mohit Shridhar et al. ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks. arXiv:1912.01734, 2019. <https://arxiv.org/abs/1912.01734>

- [28] Santhosh K. Ramakrishnan et al. BEHAVIOR: Benchmark for Everyday Household Activities in Virtual, Interactive, and Ecological Environments. arXiv:2108.03332, 2021. <https://arxiv.org/abs/2108.03332>
- [29] Stephen James et al. RLBench: The Robot Learning Benchmark and Learning Environment. arXiv:1909.12271, 2019. <https://arxiv.org/abs/1909.12271>
- [30] Jiayuan Gu et al. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills. arXiv:2302.04659, 2023. <https://arxiv.org/abs/2302.04659>
- [31] Oier Mees et al. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. arXiv:2112.03227, 2021. <https://arxiv.org/abs/2112.03227>
- [32] Bo Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. arXiv:2306.03310, 2023. <https://arxiv.org/abs/2306.03310>
- [33] Eloi Alonso et al. Diffusion for World Modeling: Visual Details Matter in Atari. arXiv:2405.12399, 2024. <https://arxiv.org/abs/2405.12399>
- [34] Dani Valevski et al. Diffusion Models Are Real-Time Game Engines. arXiv:2408.14837, 2024. <https://arxiv.org/abs/2408.14837>
- [35] Junliang Guo, Yang Ye, Tianyu He, Haoyu Wu, Yushu Jiang, Tim Pearce, and Jiang Bian. MineWorld: A Real-Time and Open-Source Interactive World Model on Minecraft. arXiv:2504.08388, 2025. <https://arxiv.org/abs/2504.08388>
- [36] Linxi Fan et al. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. arXiv:2206.08853, 2022. <https://arxiv.org/abs/2206.08853>
- [37] Michael Matthews et al. Craftax: A Lightning-Fast Benchmark for Open-Ended Reinforcement Learning. arXiv:2402.16801, 2024. <https://arxiv.org/abs/2402.16801>
- [38] Jingtao Ding et al. Understanding World or Predicting Future? A Comprehensive Survey of World Models. arXiv:2411.14499, 2024. <https://arxiv.org/abs/2411.14499>
- [39] Bohan Hou et al. World Model for Robot Learning: A Comprehensive Survey. arXiv:2605.00080, 2026. <https://arxiv.org/abs/2605.00080>
- [40] Siyin Wang et al. World Action Models: The Next Frontier in Embodied AI. arXiv:2605.12090, 2026. <https://arxiv.org/abs/2605.12090>